RESEARCH PROJECT

Parallel K-Means Clustering

Apache Spark implementation & performance evaluation

A research-inspired approach to faster clustering

Partition-level K-Means → candidate-center aggregation → distributed refinement

Chidiebere Abonyi • Anaelle Manzi • Dinh Nguyen • Abdul-Azeem Opakunle

Why parallelize K-Means?

Repeated nearest-center calculations become costly as data grows.

The challenge

Traditional K-Means iterates through assignment and center updates on the full dataset.

The opportunity

Partition-level computation can use multiple local worker threads to process data in parallel.

The question

Can execution time decrease without changing the measured clustering quality?

Research foundation

Inspired by Liu et al. (2024), IEEE ICPECA, document 10 470 970.

Paper contribution

Partition-based local clustering, combined candidate centers, and iterative global refinement.

Our contribution

A custom Python / PySpark implementation with controlled experiments and a traditional K-Means baseline.

Research-inspired adaptation — not a line-by-line replication or a reproduction of the paper's multi-node results.

Traditional K-Means in four steps

01 Initialize

Choose K starting centers

02 Assign

Find nearest center for each point

03 Update

Recalculate cluster means

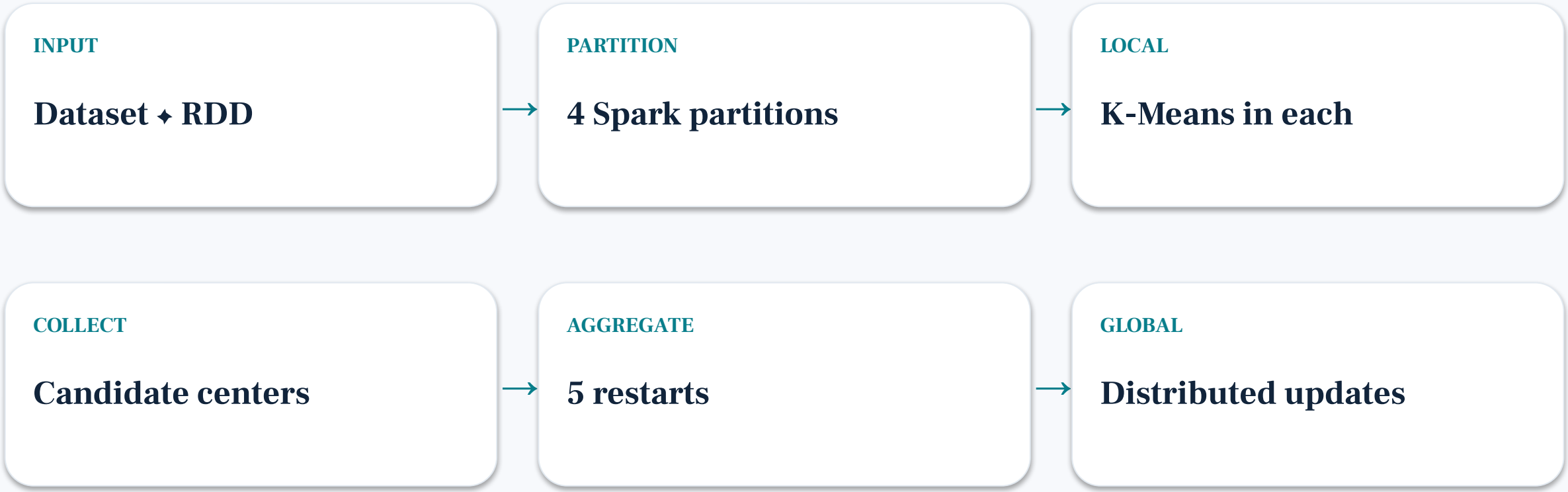
04 Repeat

Stop when convergence is reached

Objective: minimize within-cluster sum of squared errors (SSE).

Parallel K-Means pipeline

A visual summary of the nine-step implementation.



Final output: global cluster centers, SSE, convergence status, and cluster counts.

Implementation architecture

Local computation

mapPartitionsWithIndex

Custom K-Means++ initialization

Candidate-center generation

Global computation

Broadcast current centers

reduceByKey cluster statistics

Iterate until SSE-change threshold

Custom implementation — not a call to Spark MLlib's K-Means estimator.

CONTROLLED STUDY

Experimental design

42

total runs

36 Spark + 6 traditional

2

datasets

Synthetic + KDD99

3

random seeds

42, 43, 44

Each sample has 1,000 records. Spark configurations: local[1], local[2], local[4]; 2 or 4 partitions.

Historical synthetic scalability design

Historical repeated synthetic study: 10K and 50K records, 3 seeds, one machine.

DATASETS	FEATURES	CLUSTERS
10k / 50k	20	5

PARTITIONS	WORKERS	MEASURED RUNS
4	1 / 2 / 4	18

3 repetitions per configuration • seeds 42, 43, 44 • 1 unmeasured warm-up per Spark-master process

All tests ran on one Apple M2 MacBook Air (8 GB) in Spark local mode.

CONTROLLED PERFORMANCE

Within-Spark runtime speedup

1.698×

Synthetic

local[1] → local[4]

1.575×

KDD99

local[1] → local[4]

Four partitions. Speedup compares Spark local configurations—not Spark against traditional K-Means.

Synthetic clustering agreement

1.000

Spark ARI

local[4], p=4

1.000

Spark NMI

local[4], p=4

0.910

Traditional ARI

mean baseline

Strong synthetic-label agreement; initialization and restart procedures differ across implementations.

KDD99: weak label agreement

0.055

Spark ARI

local[4], p=4

0.080

Spark NMI

local[4], p=4

0.803

Spark purity

local[4], p=4

Low ARI/NMI indicate weak agreement with reference labels; purity alone can be misleading.

Traditional K-Means on small samples

0.0702 s

Synthetic

traditional mean

0.0736 s

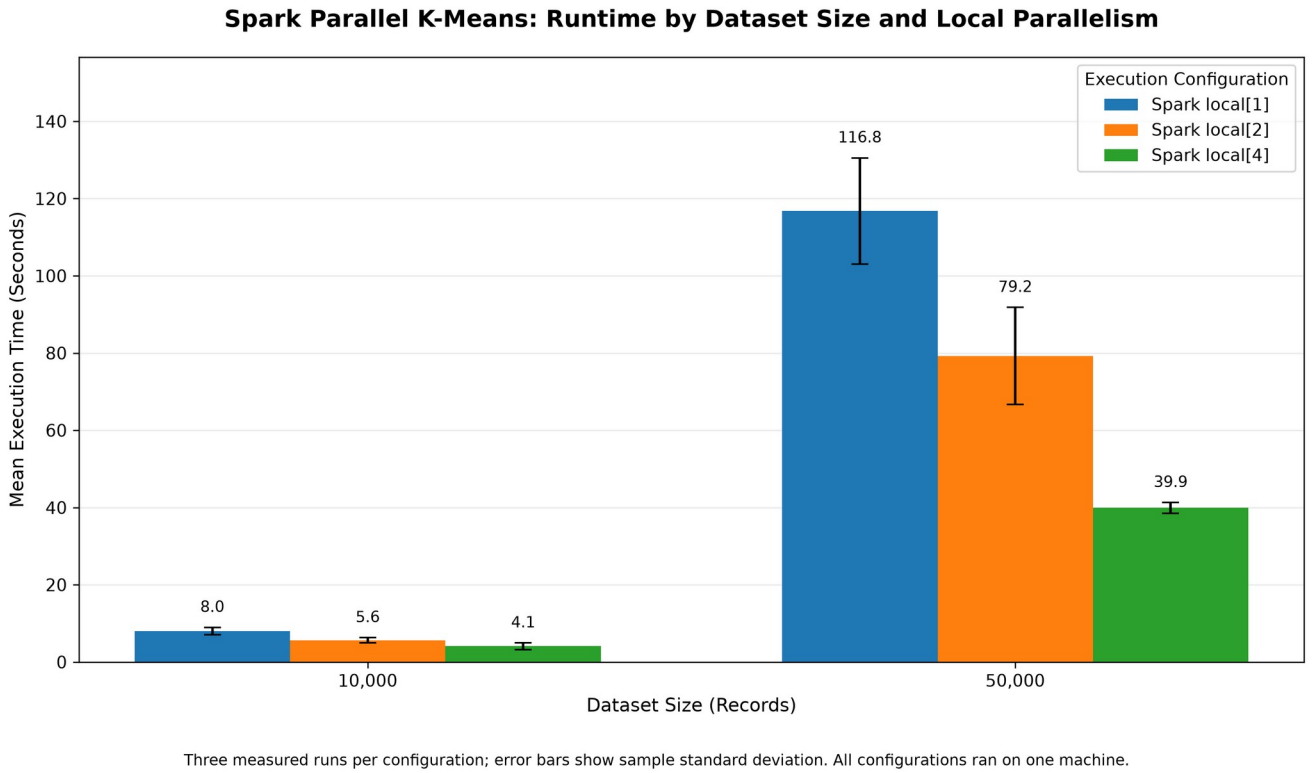
KDD99

traditional mean

Traditional K-Means was faster on 1,000-record samples. Do not infer Spark is faster at this scale.

Execution time falls with more workers

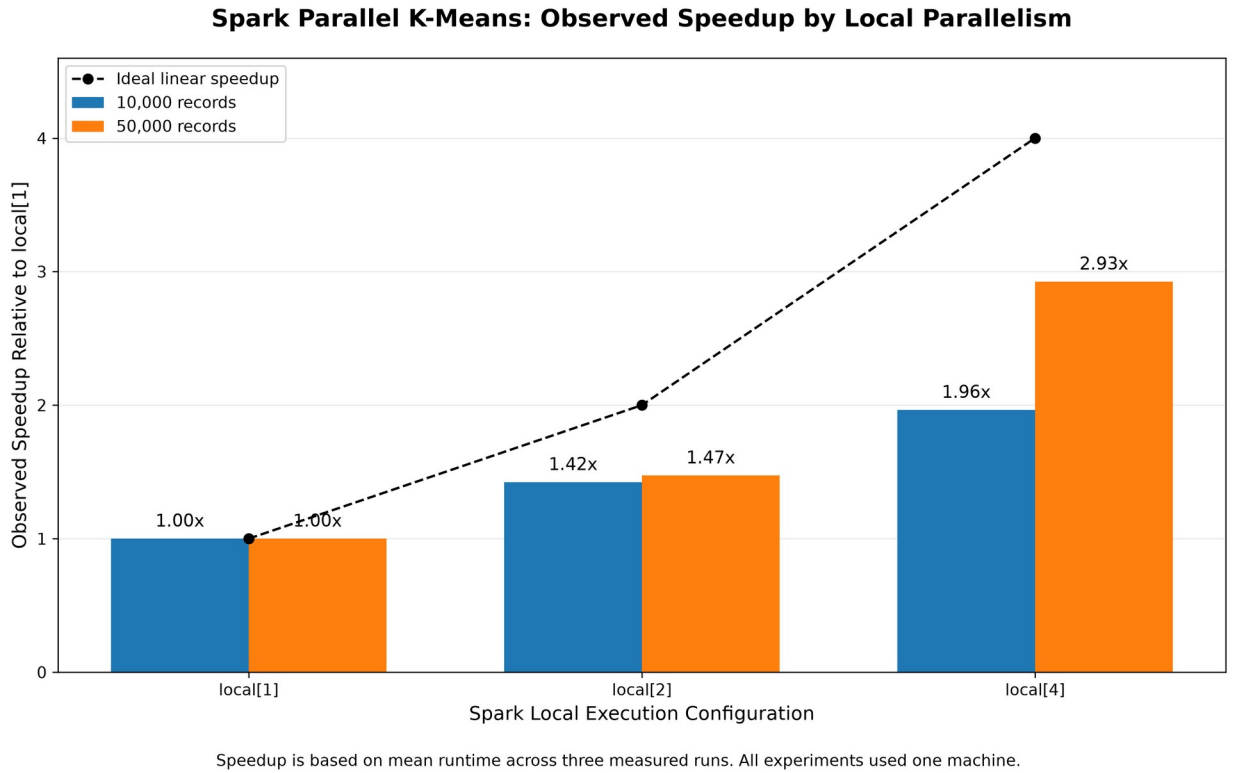
Mean total measured runtime across three measured runs.



10k: 8.033s → 4.090s | 50k: 116.765s → 39.918s

Measured speedup

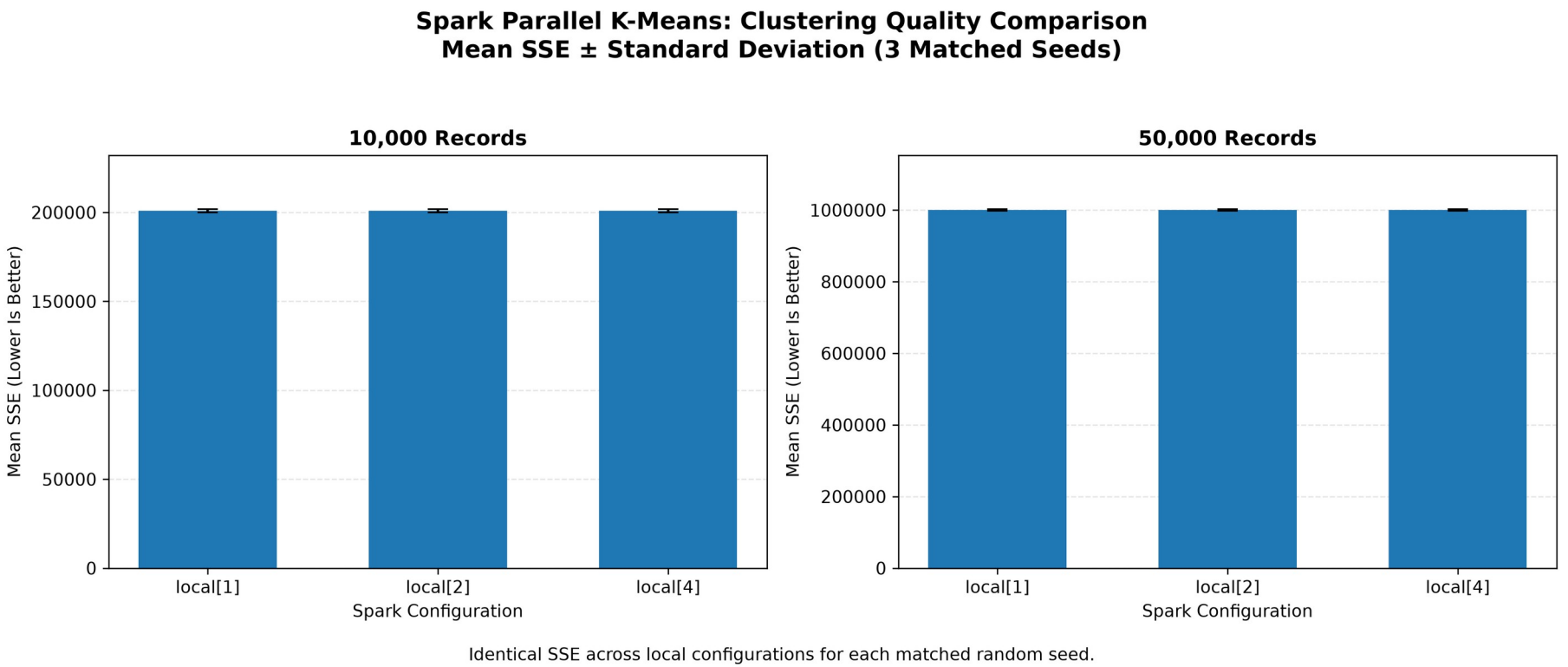
Speedup is measured relative to local[1].



local[4] achieves 1.964× at 10k and 2.925× at 50k.

Clustering quality is consistent

Mean SSE is unchanged across local worker settings at each dataset size.



10k: 200,921.733 SSE | 50k: 1,000,547.881 SSE

HISTORICAL STUDY

Separate synthetic scalability evaluation

18

historical runs

not part of the 42

10K / 50K

records

20 numerical features

2.925×

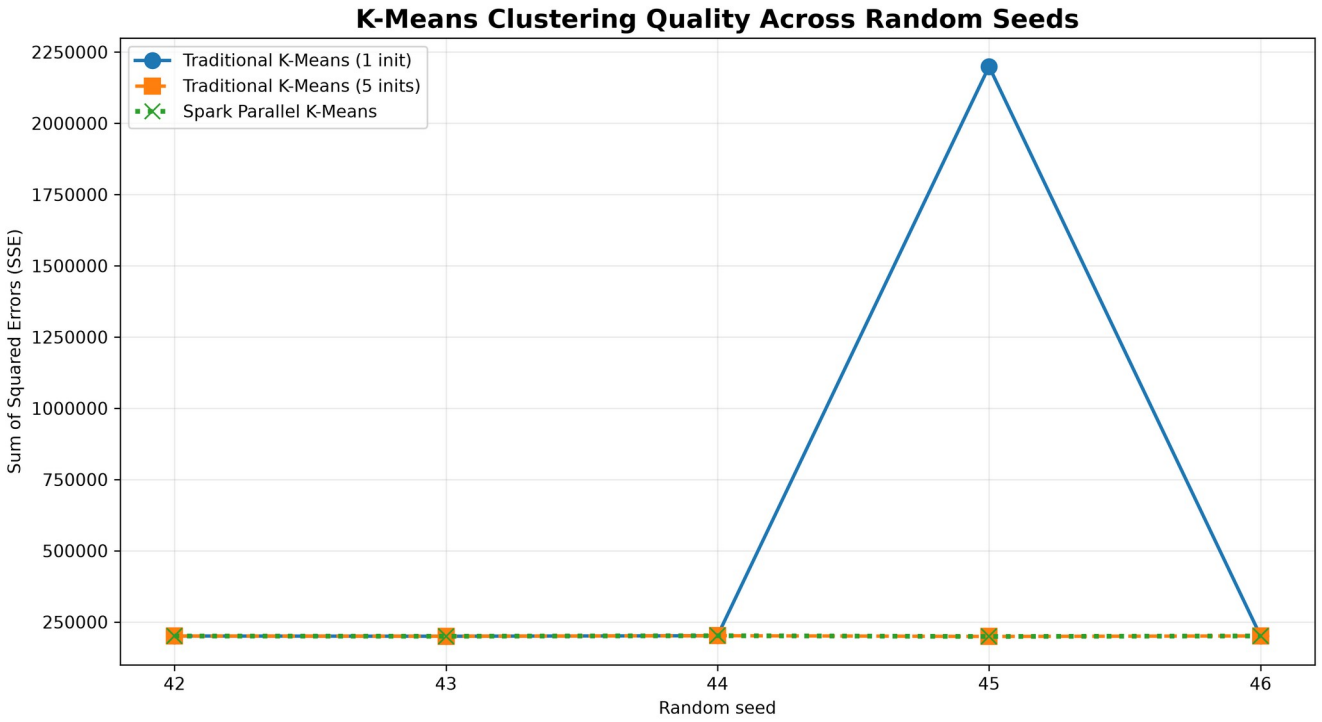
50K speedup

local[1] → local[4]

Three seeds per configuration; single-computer Spark local mode, not a physical cluster.

Traditional baseline: initialization matters

Single versus multiple initializations on the same 10k-record baseline.



10,000 synthetic records; 20 features; 5 clusters. Traditional-5 and Spark have different initialization workloads.

One seed produced higher SSE with a single initialization; multiple starts improved it.

Paper versus our implementation

Same high-level workflow, different experimental scope and design choices.

Dimension	Original research [1]	Our project
Execution	Distributed computing environment	Single-machine Spark local mode
Datasets	KDD Cup 99 and large synthetic data	10k / 50k synthetic records
Initialization	Partition-level initialization	Local K-Means++ and 5 aggregation restarts
Global update	Parallel assignment and aggregation	Broadcast + reduceByKey + SSE convergence

Raw runtime and SSE are not directly comparable across these two studies.

Scope: historical 10K/50K synthetic runs; controlled 1K synthetic + KDD99 runs.

Live demo: show the algorithm working

Small input, clear output, predictable execution time.

1 Partition

Show how records are divided into Spark partitions.

2 Cluster

Run a short Parallel K-Means demonstration.

3 Interpret

Explain centers, convergence, and SSE.

Demo scripts: scripts/demo_partitioning.py • scripts/demo_parallel_kmeans.py

Limitations of the evidence

1

computer

Spark local[n] only

3

seeds

limited uncertainty

1K

sample size

controlled evaluation

Initialization differs across baselines. Historical and controlled studies remain separate. No exact IEEE reproduction claim.

Historical scalability findings

BEST OBSERVED SPEEDUP

2.925×

RUNTIME REDUCTION

65.81%

HISTORICAL RUNS

18 / 18

Evidence

More local workers reduced runtime on both tested dataset sizes.

Boundary

These findings do not establish multi-node or real-world scalability.

What the experiments demonstrate

1.698×

Synthetic

controlled Spark speedup

1.575×

KDD99

controlled Spark speedup

0.055

KDD99 ARI

weak label agreement

Local parallelism reduced Spark runtime; traditional remained faster at 1K. Larger synthetic runs showed stronger scaling.

SUPPLEMENTARY VERIFICATION

Independent SSE & cluster-quality checks

1,000 records • 2 partitions • seed 42 • 5 clusters

Dataset / method	SSE	Purity	ARI	NMI
Synthetic • Traditional	1,032.370	1.000	1.000	1.000
Synthetic • Spark local[1]/[2]	1,032.370	1.000	1.000	1.000
KDD99 • Traditional	9,342.830	0.975	0.432	0.535
KDD99 • Spark local[1]/[2]	21,072.555	0.797	-0.005	0.011

All four Spark runs passed independent SSE, assignment, and cluster-count checks.

KDD99 purity 0.797 is near the 0.795 majority-class reference. This five-cluster check is separate from the earlier four-cluster, 42-run controlled study. Spark local[n] uses one computer.

Thank you — Questions?

Parallel K-Means clustering using Apache Spark

Historical synthetic local[4] speedup: 2.925× at 50,000 records — with consistent mean SSE across local settings

[1] H. Liu, Y. Bai, Z. Chen, and Z. Zhang, “Big data clustering method based on parallel K-means,” IEEE ICPECA, 2024. DOI: 10.1109/ICPECA60 615.2024.10470 970.

[2] S. P. Lloyd, “Least squares quantization in PCM,” IEEE Trans. Inf. Theory, 1982.

[3] D. Arthur and S. Vassilvitskii, “k-means++,” SODA, 2007.

[4] B. Bahmani et al., “Scalable K-Means++,” PVLDB, 2012.

[5] Apache Software Foundation, “RDD Programming Guide,” Spark Documentation.